**CY Cergy-Paris Université**

# RAPPORT

pour le projet de Génie Logiciel
**Double-Licence Mathématiques Informatique, 2ème Année**

sur le sujet

# Simulation de vie urbaine


rédigé par

**Nur CHOUAREF, Léanore JEAN, Mohamed LOUKILI**

encadré par

**Tianxiao LIU**





Avril 2026

# Table des matières

**1 Introduction** **4**
1.1 Contexte du projet . . . 4
1.2 Objectif du projet . . . 4
1.3 Organisation du rapport . . . 4

**2 Spécification du projet** **6**
2.1 Notions de base et contraintes du projet . . . 6
2.2 Fonctionnalités attendues du projet . . . 8

**3 Conception et mise en œuvre** **9**
3.1 Architecture globale du logiciel . . . 9
3.2 Classes de données . . . 10
3.3 IHM Graphique . . . 12
3.3.1 Menu et configuration initiale . . . 12
3.3.2 Interface de jeu : Dashboard et SidePanel . . . 13
3.4 Conception des traitements (processus) . . . 14
3.4.1 Mise à jour des besoins et gestion du cycle de vie . . . 14
3.4.2 Dynamiques sociales et algorithme de cohabitation . . . 15
3.4.3 Génération d'événements et progression de difficulté . . . 15

**4 Manuel utilisateur** **16**
4.1 Lancement et Menu principal . . . 16
4.2 Interface de jeu et Dashboard . . . 18
4.3 Interaction et gestion des Sims . . . 20

**5 Déroulement du projet** **22**
5.1 Réalisation du projet par étapes . . . 22
5.2 Répartition des tâches, communication et intégration . . . 23

**6 Tests du projet** **25**

**7 Conclusion et perspectives** **27**
7.1 Résumé du travail réalisé . . . 27
7.2 Améliorations possibles du projet . . . 27

# Table des figures

1 Schéma de l'architecture global du logiciel . . . 9
2 Diagramme de classe donnée simplifié . . . 11
3 Interface du menu principal . . . 12
4 Interface des règles du jeu . . . 12
5 Interface du choix du nom de la ville . . . 13
6 Vue de l'interface principale combinant le Dashboard et le SidePanel . . . 13
7 Détails des besoins vitaux . . . 14
8 La fenêtre d'accueil du menu principal avec ses trois boutons de navigation. . . . 16
9 Le panneau de personnalisation du nom de la ville, avec le champ de saisie pré-rempli. 17
10 Le panneau des règles du jeu, accessible depuis le menu principal. . . . 17
11 Vue générale de la fenêtre principale au jour 2 : la carte de la ville à gauche avec les Sims visibles, et le tableau de bord latéral à droite affichant les informations de Marie. 18
12 Vue de la fenêtre principale avec Marie sélectionnée : le `SidePanel` affiche sa routine, ses relations amicales en cours de formation et le journal des événements. . . . 19
13 Fenêtre de détail des barres de vie de Marie : Faim à 98 %, Physique à 94 % et Mental à 86 %. . . . 20

14 Fenêtre des alertes Sims en cours au jour 3, niveau 2 : aucune alerte active. On distingue en arrière-plan le `SidePanel` de Marie avec sa routine, ses relations amicales et le bouton **Recommencer le jeu**. . . . . 20

## Liste des tableaux

1 Répartition des tâches entre les membres de l'équipe . . . . . 24
2 Tests d'utilisation . . . . . 25
3 Tests de robustesse . . . . . 25
4 Tests de performance . . . . . 26

## Remerciements

Nous, auteurs du projet Urbain, à savoir Nur CHOUAREF, Léanore JEAN et Mohamed LOUKILI, tenons à remercier vivement notre professeur, Tianxiao LIU, pour son accompagnement et ses précieux conseils tout au long de ce travail.

Nous le remercions notamment pour la méthode de travail proposée, qui nous a permis d'adopter une approche plus dynamique et structurée, ainsi que pour la méthodologie détaillée concernant la réalisation d'un projet dans toutes ses étapes.

Ce projet nous a également permis de nous appuyer sur des exemples concrets de programmes, constituant une véritable source d'inspiration pour développer des projets plus professionnels dans cette matière.

Nous soulignons également l'importance des conseils reçus pour la préparation de la présentation, notamment en ce qui concerne la conception des diapositives, qui nous seront utiles pour nos futurs projets.

# 1 Introduction

## 1.1 Contexte du projet

**Choix du projet : "Urbain"** Dans le cadre de l'UE Génie Logiciel, notre équipe a choisi de développer *Urbain*. Ce projet consiste à modéliser un environnement citadin où des entités autonomes doivent coexister et survivre malgré les aléas du quotidien. Ce choix repose sur notre volonté d'explorer la complexité des systèmes multi-agents et de mettre en œuvre une architecture logicielle capable de gérer des interactions sociales et biologiques en temps réel.

**Une approche de la vie en société** Contrairement à une simple animation graphique, le projet *Urbain* impose une gestion rigoureuse d'états internes. Chaque habitant est défini par des variables de besoins vitaux, faim, santé physique et mentale, qui varient selon son activité, ses relations et les évènements qui l'impactent. Reproduire ces dynamiques nécessite une séparation claire entre la logique métier, telle que la gestion des besoins ou la planification des routines, et l'interface utilisateur.

**Motivations techniques et pédagogiques** Nous avons retenu ce sujet pour sa richesse en termes de conception orientée objet. Le projet nous oblige à structurer notre application autour de gestionnaires spécialisés (*Managers*) chargés de traiter le caractère aléatoire des événements et la gestion des données sociales. Au-delà du code, cette expérience nous plonge dans les pratiques professionnelles du génie logiciel : collaboration via des outils de versionnage (GitHub), documentation des composants et tests d'intégration, afin d'assurer la stabilité d'une simulation où le hasard joue un rôle central.

## 1.2 Objectif du projet

Ce projet a pour objectif de simuler une société au sein d'une ville avec laquelle l'utilisateur peut interagir. Chacun des habitants possède une routine et des habitudes propres ; c'est, en substance, un jeu de gestion de vie intégrant des événements aléatoires qui bousculent individuellement ou collectivement le quotidien des membres de la société.

L'utilisateur doit apprendre à gérer des situations complexes dans l'intérêt du bien-être de tous les Sims. L'objectif ultime est d'atteindre un équilibre permettant de bâtir une société saine, tout en évitant les décès qui pourraient mener à une fin de partie. La progression est marquée par un système de niveaux récompensant la survie prolongée des habitants.

## 1.3 Organisation du rapport

Ce rapport est structuré de manière à offrir une vue d'ensemble cohérente du projet, de sa spécification technique à sa réalisation concrète.

**Section 2 - Spécification du projet :** Détaille le cahier des charges, les contraintes de l'environnement urbain, ainsi que les portraits et les routines des six Sims initiaux. Les mécaniques de jeu (besoins, événements, progression) y sont également décrites.

**Section 3 - Conception et mise en œuvre :** Analyse l'architecture globale du logiciel (données, managers, repositories), le diagramme de classes de données et l'organisation de l'IHM graphique (Dashboard, SidePanel). Les trois algorithmes principaux régissant la vie des Sims y sont notamment clarifiés.

**Section 4 - Manuel utilisateur :** Présente les instructions pour configurer et piloter la simulation, illustrées par des captures d'écran des différentes fenêtres de l'application.

**Section 5 - Déroulement du projet :** Retrace les étapes clés du développement par points d'avancement successifs, et détaille la répartition des tâches au sein de l'équipe ainsi que les outils de communication et d'intégration utilisés.

**Section 6 - Conclusion et perspectives :** Récapitule le travail accompli au regard des objectifs initiaux et propose des axes d'amélioration concrets pour enrichir la simulation dans de futures versions.

# 2 Spécification du projet

Nous avons présenté l'objectif du projet dans la section 1. Dans cette section, nous présentons la spécification de notre logiciel réalisé.

## 2.1 Notions de base et contraintes du projet

**Ville :** La ville constitue le cadre dans lequel le jeu se déroule. Elle est fixe et ne change pas au cours de la partie. La ville est composée des bâtiments transparents et on les verra par point de vue aérien (vue dessus). On y trouve différent type de lieux :

- Les lieux habitation donc un quartier résidentiel composée de 3 maisons indépendantes avec un jardin commun et un immeuble ou plusieurs personnes habitent.
- Les lieux de travail donc une, usine, un centre commercial et aussi une boulangerie.
- La mairie de la ville où le maire travail.
- Lycée et une école primaire où la fille et l'adolescent étudient
- Les lieux de loisir comme une piscine une salle de sport qui permet de garder un bon état de santé physique mais aussi le cinéma aussi permet d'augmenter la santé mentale.
- L'hospital, en cas de maladie ou urgence.

Entre les bâtiments il y a des routes toutes parallèles où sinon perpendiculaires de façon à obtenir une ville à plan romain où en damier. Et pour conclure les espace vide ou non construits seront rempli par des espace vert pour rendre la ville plus jolie. Un Sim : c'est un habitant de la ville caractérisé pas son âge, profession ou occupation une routine et un état de bien être mentale et physique.

**Les 6 sims :**

- Sim 1 : C'est un homme, adulte, qui a 37 ans. Il est ouvrier dans une usine d'électro-ménager et travaille de 8h00 à 12h00 puis de 14h00 à 18h00 toute la semaine soit le lundi, mardi, mercredi, jeudi et vendredi. Sur son temps libre, il va à la salle de sport. S'il n'a pas sa dose de musculation tous les jours, il peut se sentir mal. Le week-end, il en profite pour faire des sorties telles que : aller au cinéma ou bien aller au centre commercial.
- Sim 2 : C'est une femme âgée de 74 ans. Elle est à la retraite et passe la plupart de son temps chez elle. Elle s'occupe de son jardin quotidiennement, tous les matins, et sort tous les jours vers 16h00 se promener dans le parc pour faire sa marche quotidienne. Le dimanche matin, elle va à la piscine faire son cours d'aquagym. Attention à ne pas le rater car un manque d'activité physique peut être très fatal !
- Sim 3 : C'est un adolescent de 17 ans. Il est au lycée toute la semaine (lundi, mardi, mercredi, jeudi et vendredi) de 9H00 à 17h00. Lorsqu'il rentre des cours et qu'il est chez lui, il est principalement dans sa chambre en train de jouer aux jeux vidéo. Il est addict à ces jeux et ne peut pas s'en passer. Parfois il reste très tard sur sa console, attention à bien lui rappeler d'aller dormir car c'est aussi l'année de son bac, il doit donc garder un rythme adéquat à ses études. Le week-end il fait au moins une sortie comme aller au parc ou bien au cinéma. S'l ne sort pas au moins une fois le week-end, il risque de se renfermer sur lui-même.
- Sim 4 : C'est un enfant, une petite fille qui a 8 ans. Les lundis, mardis, jeudis et vendredis, elle est à l'école primaire de 8h30 à 16h30. Les mercredis, elle passe tout l'après-midi, de 15h00 à 18h00 au parc et les week-end, elle fait des sorties comme aller au cinéma, au centre commercial au bien au parc.
- Sim 5 : C'est une femme adulte de 34 ans. Elle est boulangère et travaille toute la semaine, du lundi au samedi de 6h30 à 14h00 et les après-midis, elle fait souvent une activité les mardis et jeudis après-midi telle qu'aller au cinéma. De plus, elle va au moins deux fois par semaine à la salle de sport, sur son temps libre.
- Sim 6 : C'est un homme, adulte, de 50 ans. Il est le maire de la ville et il va travailler à la mairie tous les jours, pendant 5 heures entre 11h00 et 16h00. Son emploi du temps est assez libre et les moments où il n'est pas à la mairie, il passe voir comment va sa ville. C'est à dire qu'il va au moins dans deux endroits parmi l'école primaire, le lycée, la boulangerie, la salle de sport,

l'usine, la piscine et l'hôpital. Il est diabétique et à besoin de prendre son traitement tous les jours, à la même heure sinon il risque d'être dans un état critique et de se retrouver à l'Hospital.

**Routine :** Chaque Sim a une routine bien définie qui suit en automatique. La routine de chaque Sim est définie dans le paragraphe précédent.

**Les besoins principaux :** Les besoins principaux sont représentés par une barre qui varie entre 0% et 100%, elle permet de connaître l'état du Sim au cours du jeux. Entre 75-100% le Sim est efficace et rapide, entre 50%-75% le Sim est normal, entre 25%-50% le Sim est fatigue lent, entre 0%-25% maladie physique ou psychologique. Lorsqu'une des trois barres atteignent un niveau trop bas, le Sim subit des effets négatifs affectant sa routine. Les trois besoins sont les suivant :

- Le bien-être physique : Il est influencé par l'activité physique (donc le sport excessif le travail et les déplacements), le repos et les soins médicaux. Un niveau trop bas peut conduire à des maladies qui nécessite une visite à l'hospital et implique aussi limiter les actions possibles comme aller au travail. Cas où le bien-être physique est entre 0% et 25% : le Sim est malade tant qu'il n'est pas soigné, le bien-être physique continue à diminuer et les conséquences sont les suivantes : -Le Sim ne peut plus travailler ni aller à l'école et les activités physique sont bloquées.
- Le bien être mental : représente l'état psychologique du Sim. Il est influencé par l'activité de loisir et les interactions sociales ou des évènements (spécifie en suite). Pour augmenter le bien être mental l'utilisateur doit lui faire faire l'activité du loisir du Sim cette à dire comme aller au Cinéma, Parc, Centre commercial. Cas où le bien-être mental est entre 0% et 25% : Le Sim est psychologiquement en détresse et les conséquences sont les suivantes : -Ne suit plus la routine et peut rester à ne rien faire. -La seul solution ce sont les loisirs comme le cinéma, le parc, la salle et aussi du repos.
- La faim : Représente la satiété du Sim. Influencé par la nourriture consommée. Cas où la faim est entre 0% et 25% : le Sim est affamé ce qui commence à réduire de 5% tous les heure l'état physique et l'état mental. La seule solution c'est manger.

**Relations amicales :** chaque Sim peut développer des liens d'amitiés avec un autre Sim, l'amitié se forme quand les deux Sim passent 20 heures ensemble dans le même lieux ou bâtiments dans les 30 dernier jour. Lorsqu'une relation amicale existe, elle augmente le bien être mental du Sim de 5% à chaque heure qu'il passe dans le même bâtiment.

**Temps dans le jeu :** dans le jeu on compte les journées, on distingue aussi les 7 jours de la semaine et chaque jour correspond à 10 minutes de jeu.

**Niveau :** Lorsque la semaine s'écoule sans qu'aucun Sims ne meurt, on augmente d'un niveau. Lorsqu'on atteint un certain niveau, un nouveau Sims emménage dans la ville.

**Evènements majeurs :**

- Deuil : Dans le cas d'un décès de Sim, les Sims qui avaient une relation quelconque avec lui seront confrontés aux deuils, il faudra donc particulièrement les surveiller.
- Evènements quotidiens : Il s'agit des évènements qui bousculent le bien être des Sims, tel que la salle de sport fermée pour les sportifs.
- Urgence médicale : Le plus haut niveau d'évènement est l'urgence médicale qui nécessite d'être traitée rapidement sinon elle entraine la mort du Sims.
- Mort d'un Sim : Elle est rencontrée quand le bien-être mental et le bien-être physique sont tous les deux entre 0% et 25% pendant 3 jour, la seule solution c'est l'Hospital. Ou sinon lorsque la faim est entre 0 et 25% pourcent pendant trois jours.
- Fin partie : Lorsque 3 Sims meurent, la partie est terminée.

**Evènements quotidiens :**

- Saut de repas : Le Sim saute un repas à cause d'un loisir ou la surcharge de travail et donc la barre de faim elle diminue de 25%.
- Maladie : Le Sim tombe malade (grippe, rhume, angine...), la barre de bien être physique diminue à 10% toute les heures tant qu'il n'est pas guéri, et le bien être moral diminue de 25%.
- Solitude : Les Sim ayant des amis se sentent seuls après avoir passé 3 jours sans les voir, leur bien être mental diminue de 30%.
- Traitement : Les Sim ayant des maladies chroniques doivent prendre régulièrement leur traitement, sinon le bien être physique diminue de 50%.
- Travaux : Les établissements de divertissement/sport peuvent être temporairement indisponible pour cause de travaux, les Sims impactés par ces fermetures voient leur bien être mental diminué de 15%.
- Le baccalauréat : Dans le cas où l'étudiant rate son bac, son bien être mental chute de 60%.

## 2.2 Fonctionnalités attendues du projet

Le logiciel "Urbain" a été conçu pour offrir une simulation systémique complexe où chaque entité (Sim) possède une autonomie propre tout en restant sous l'influence directe de l'utilisateur et de l'environnement. Les fonctionnalités implémentées se divisent en quatre axes majeurs :

**Gestion dynamique des agents et cycles de vie :** Au cœur du projet se trouve le moteur de besoins. Chaque Sim est régi par des jauges de faim, de santé physique et de bien-être mental qui évoluent en temps réel. Le logiciel ne se contente pas de soustraire des points ; il calcule l'impact de chaque lieu (travail, loisirs, domicile) sur ces jauges. Une fonctionnalité critique est la gestion de l'état "survie" : lorsqu'un Sim entre en zone critique (moins de 25%), un sursis de trois jours s'active, au terme duquel le décès est prononcé si aucune action corrective n'est entreprise.

**Intelligence de navigation et routines :** La simulation repose sur une automatisation poussée des déplacements. Le système interprète les routines hebdomadaires (lundi-vendredi) et de week-end pour téléporter ou déplacer les agents vers leurs objectifs respectifs (usine, mairie, école, etc.). Cette autonomie est complétée par une fonctionnalité d'interactivité manuelle : l'utilisateur peut à tout moment reprendre le contrôle d'un Sim par une interaction à la souris sur la carte pour pallier une urgence.

**Moteur d'interactions sociales :** Le logiciel intègre une dimension relationnelle gérée par un algorithme de cohabitation. Le système suit le temps passé en commun par les Sims dans un même bâtiment. À l'atteinte du seuil de 20 heures cumulées, une relation amicale est formalisée dans le dépôt de données (*Repository*). Cette amitié devient une fonctionnalité active car elle génère un bonus de santé mentale automatique dès que les Sims se croisent à nouveau.

**Système d'événements et progression :** Pour briser la monotonie de la routine, un gestionnaire d'événements injecte des aléas (maladies, fermetures de bâtiments, deuils). Enfin, le logiciel gère la progression globale de la ville via un système de niveaux : une semaine sans décès entraîne une montée de niveau, augmentant la complexité de la simulation par l'ajout de nouveaux habitants, renforçant ainsi l'aspect stratégique du jeu.

# 3 Conception et mise en œuvre

Cette section présente les choix techniques qui ont guidé le développement du jeu. Après une phase de conception, nous présentons l'organisation générale du logiciel, les classes de données, l'interface graphique, ainsi que les solution pour le traitement qui sont le noyau de notre jeu.

## 3.1 Architecture globale du logiciel

Le logiciel est organisé en trois couches distinctes qui, comme on peut le voir dans le schéma 1, communiquent dans un sens unique : la GUI interroge le traitement, le traitement lit et modifie les données.

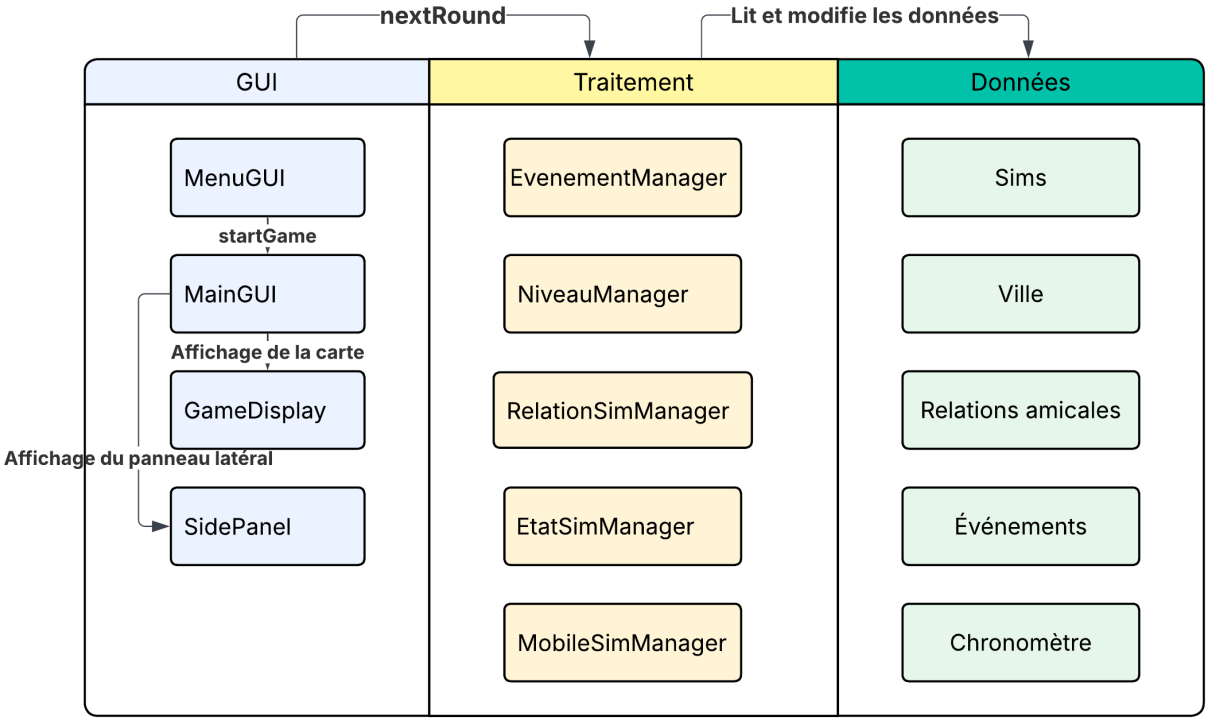



FIGURE 1 – Schéma de l'architecture global du logiciel

**GUI** contient les classes Swing responsables de l'affichage : MainGUI est la fenêtre principale qui cadence la simulation à chaque tick, GameDisplay dessine la carte et les Sims, SidePanel affiche les informations des personnages, et MenuGUI gère l'écran d'accueil.

**Le traitement** constitue le cœur fonctionnel du jeu. GameBuilder initialise l'ensemble des composants au démarrage. Quatre managers spécialisés prennent ensuite en charge chaque aspect de la simulation : MobileSimManager déplace les Sims selon leurs routines, EtatSimManager met à jour les barres de vie, EvenementManager déclenche les événements aléatoires, et RelationSimManager gère l'évolution des amitiés. NiveauManager ajuste la difficulté globale. Trois repositories (EvenementsRepository, RelationsRepository, DerniersEvenementsRepository) centralisent les données partagées entre les managers, selon le pattern Singleton.

**Les données** regroupe toutes les entités du jeu : les Sims et leurs routines, les bâtiments et la ville, les événements, les relations amicales, et le chronomètre.

### 3.2 Classes de données

Le diagramme de classe donnée final, dans la figure 2, représente l'ensemble des classe donnée utilisées dans le jeux et leurs relations. La classe principale et avec le plus de donnée est Sim. Chaque Sim possède une position dans la mappe et donc un Bloc, deux routines distantes pour la semaine et pour le Weekend, trois compteurs pour la faim, le physique et le mentale ansi que ses Etats.

La classe Routine, associée a chaque début d'heure bâtiment. Contrairement à la conception initiale, où une routine n'était qu'une succession d'événements, cette solution nous a permis définir un journée et donc pouvoir avoir un cycle. Les Bâtiments et la Ville forment l'environnement dans lequel évoluent les Sims : chaque bâtiment possède un nom, ses dimensions pour définir son espace, et la Ville regroupe l'ensemble des bâtiments.

Les interactions sociales et les événements sont représentés par les classes RelationAmicale et Evenement. La classe RelationAmicale, permet de suivre le niveau d'amitié entre deux Sims, remplaçant les simples attributs statiques prévus initialement. La classe Evenement relie son impacte à une des trois barre de vie du sim. Par rapport à la conception initiale, de ajustements ont été apportés pour répondre aux besoins concrets du jeu : la création de la classe Etats pour enregistrer les impacts et seuils, la transformation des routines pour gérer les déplacements dans les bâtiments, l'ajout des dimensions des bâtiments, et la création d'une classe dédiée pour gérer les relations d'amitié.

Dans l'ensemble, le diagramme de classe reflète nos choix initiaux. Il a évolué au fur et à mesure en fonction des contraintes et des besoins rencontrés. L'architecture finale propose une organisation claire entre les données liées à l'environnement, comme la mappe et les bâtiments, celles des personnages, comme les Sims, leurs états et leurs routines, ainsi que celles des interactions, comme les relations et les événements.

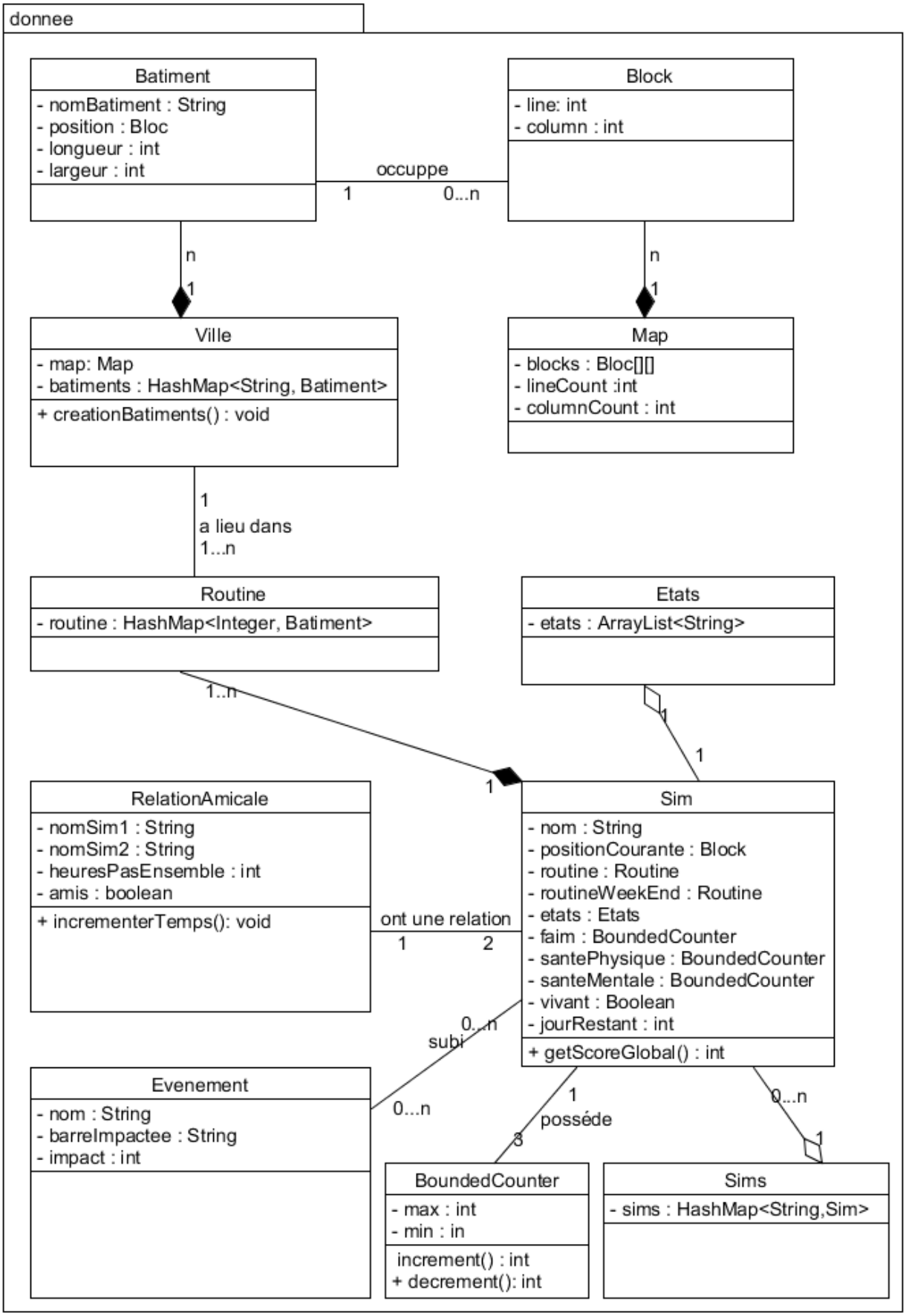



Figure 2 – Diagramme de classe donnée simplifié

## 3.3 IHM Graphique

L'interface homme-machine (IHM) de notre simulateur a été développée avec la bibliothèque Java Swing. L'objectif principal était de fournir une vue d'ensemble claire de la ville tout en permettant un accès rapide aux statistiques détaillées de chaque Sim. L'interface se décompose en deux modules majeurs : le menu de configuration et l'interface de jeu principale.

### 3.3.1 Menu et configuration initiale

Avant de lancer la simulation, on utilise la classe `MenuGUI` pour préparer la partie, notamment pour choisir le nom de la ville9 et lire les règles10. Nous avons organisé les menus pour qu'ils s'affichent tous dans la même fenêtre sans avoir besoin d'en ouvrir de nouvelles. Cela permet de passer de l'écran d'accueil aux paramètres de façon fluide, tout en gardant une interface stable et propre dès le début du jeu.




FIGURE 3 – Interface du menu principal

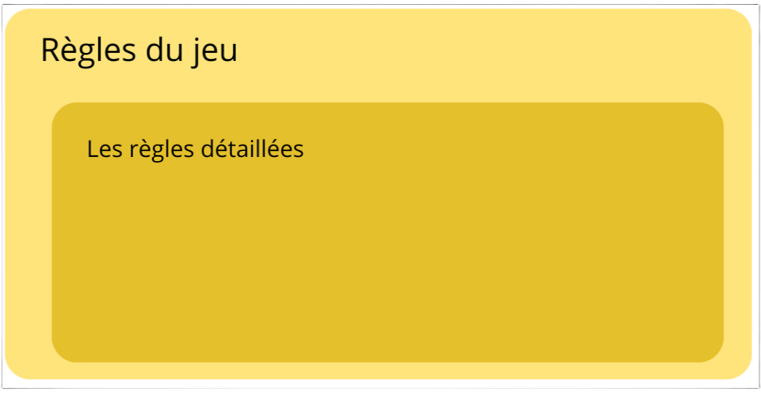



FIGURE 4 – Interface des règles du jeu

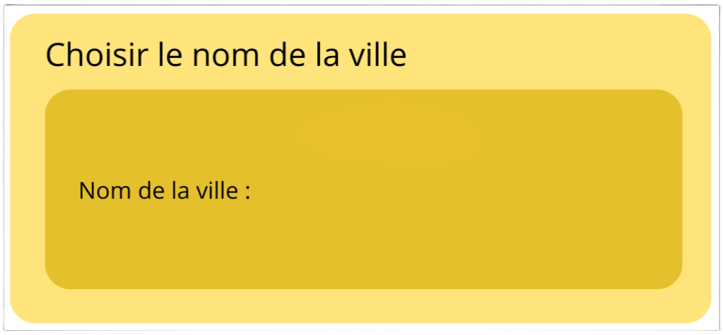



FIGURE 5 – Interface du choix du nom de la ville

### 3.3.2 Interface de jeu : Dashboard et SidePanel

Une fois la simulation lancée, l'écran principal s'affiche avec toutes les commandes nécessaires au jeu. On y trouve deux zones qui fonctionnent ensemble : le `Dashboard` au centre, pour voir la ville et les déplacements des Sims, et le `SidePanel` sur le côté pour suivre toutes les statistiques. Les deux parties restent synchronisées en temps réel grâce à une mise à jour automatique qui s'effectue à chaque seconde de la simulation.

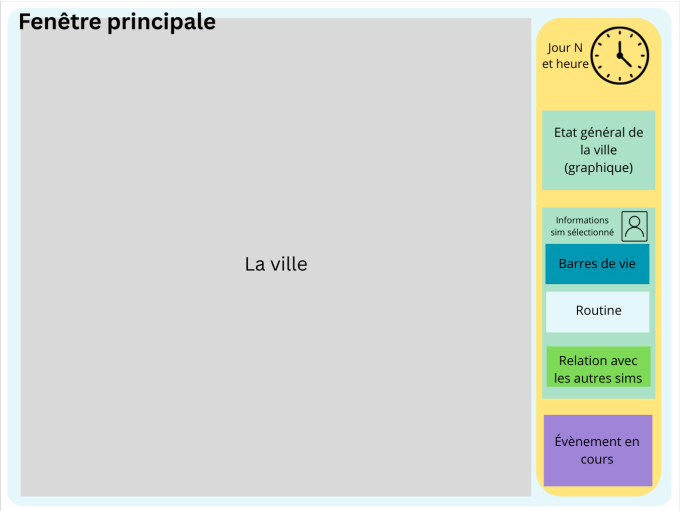



FIGURE 6 – Vue de l'interface principale combinant le Dashboard et le SidePanel

Le Dashboard, géré par `GameDisplay`, utilise une `PaintStrategy` pour dessiner les bâtiments et les agents. En parallèle, le SidePanel extrait les données des différents *Managers* pour mettre à jour les `JProgressBar` de besoins13 (Faim, Physique, Mental) et le journal des événements. Cette zone latérale permet également de consulter les relations amicales et les routines horaires du Sim sélectionné.




FIGURE 7 – Détails des besoins vitaux

## 3.4 Conception des traitements (processus)

La dynamique de la simulation repose sur une boucle de contrôle principale (*NextRound*) située dans la classe `MainGUI`. À chaque itération du thread, le moteur de jeu coordonne les différents gestionnaires pour mettre à jour l'univers urbain. Cette section détaille les trois processus fondamentaux régissant la vie autonome des Sims.

### 3.4.1 Mise à jour des besoins et gestion du cycle de vie

Le traitement le plus sensible est celui opéré par `EtatSimManager`. Ce dernier calcule les jauges de faim, de santé physique et mentale en fonction du temps écoulé et de la localisation de chaque Sim dans la ville. Lorsqu'un Sim entre en zone critique (jauge inférieure à 25%), un processus de sursis s'enclenche, formalisé dans l'algorithme 1.

**Algorithm 1** Gestion de l'état critique et du décès

**Require:** Un Sim $S$ vivant
**Ensure:** Mise à jour du sursis ou suppression du Sim
  **if** $S$ possède l'état *"faim"*, *"triste"* ou *"fatigue"* **then**
    $jourCourant \leftarrow Chronometer.getDay()$
    **if** $jourCourant \neq S.getDernierJourCritique()$ **then**
      $S.setDernierJourCritique(jourCourant)$
      $S.getJourRestant().decrement()$ //Sursis initialisé à 3 jours
      **if** $S.getJourRestant().getValue() == 0$ **then**
        $simsToKill.add(S)$ //Marquage pour suppression sécurisée
        $declencherDeuil(S)$ //Impact de 30 points sur le mental des amis
      **end if**
    **end if**
  **else**
    $S.getJourRestant().reset()$ //Retour à la normal
    $S.setDernierJourCritique(-1)$
  **end if**

Ce mécanisme de sursis est essentiel : il laisse à l'utilisateur une fenêtre d'intervention de trois jours pour corriger la situation d'un Sim en danger, sans sanctionner immédiatement une dégradation passagère des besoins.

### 3.4.2 Dynamiques sociales et algorithme de cohabitation

Le `RelationSimManager` traite les interactions sociales en vérifiant périodiquement la présence simultanée de plusieurs Sims dans un même bâtiment. L'algorithme 2 simule l'émergence de liens affectifs fondés sur le temps partagé : passé le seuil de 20 heures cumulées, une relation amicale est formalisée dans le `RelationsRepository` et génère un bonus de bien-être mental automatique dès que les deux Sims se retrouvent dans le même lieu.

**Algorithm 2** Mise à jour des relations sociales

**Require:** Liste des Sims actifs $L$, Ville $V$
**Ensure:** Évolution de l'amitié et bonus de moral
  **for all** paires de Sims $(A, B)$ dans $L$ **do**
    **if** $A$ et $B$ sont dans le même `Batiment` **then**
      $rel \leftarrow RelationsRepository.getOuCreer(A.nom, B.nom)$
      $rel.incrementerTemps()$
      **if** $rel.sontAmis()$ **then**
        Appliquer 5 incréments de moral à $A$ et $B$
      **end if**
    **end if**
  **end for**

### 3.4.3 Génération d'événements et progression de difficulté

Enfin, le moteur intègre une logique de difficulté évolutive pilotée par le `NiveauManager` et le `EvenementManager`. L'algorithme 3 montre comment le système injecte des aléas pour rompre la monotonie des routines automatiques et renforcer l'aspect stratégique du jeu : à chaque niveau supplémentaire, le nombre d'événements quotidiens augmente, rendant la gestion de la population progressivement plus exigeante.

**Algorithm 3** Génération et application d'un événement aléatoire

**Require:** `Sims`, `EvenementsRepository`
**Ensure:** Application d'un malus sur les statistiques d'un Sim choisi au hasard
  $S \leftarrow getRandomSim()$
  $E \leftarrow getRandomEvent()$
  $barre \leftarrow E.getBarreImpactee()$
  $valeur \leftarrow E.getImpact()$
  **if** $barre ==$ "$faim$" **then**
    **for** $i \leftarrow 0$ **to** $valeur - 1$ **do**
      $S.getFaim().decrement()$
    **end for**
  **end if**
  **if** $barre ==$ "$physique$" **then**
    **for** $i \leftarrow 0$ **to** $valeur - 1$ **do**
      $S.getPhysique().decrement()$
    **end for**
  **end if**
  **if** $barre ==$ "$mental$" **then**
    **for** $i \leftarrow 0$ **to** $valeur - 1$ **do**
      $S.getMental().decrement()$
    **end for**
  **end if**
  $DerniersEvenementsRepository.add(S.nom + "- > " + E.toString())$

# 4 Manuel utilisateur

Cette section est dédiée au manuel utilisateur.

## 4.1 Lancement et Menu principal

Au démarrage de l'application, l'utilisateur est accueilli par la classe `MenuGUI`, qui constitue le point d'entrée de la simulation. Cette fenêtre d'accueil présente trois options principales disposées verticalement, permettant de configurer la partie avant de lancer la simulation. La figure 8 illustre cette fenêtre d'accueil telle qu'apparaît au lancement du programme.




FIGURE 8 – La fenêtre d'accueil du menu principal avec ses trois boutons de navigation.

**Lancer la simulation** Le bouton **Jouer** constitue l'action centrale du menu. Son activation ferme la fenêtre d'accueil et instancie la fenêtre principale du jeu via la classe `MainGUI`. Le passage entre les deux fenêtres est gérée par un thread dédié au menu, dont la boucle de contrôle repose sur un mécanisme de type *switch/case* : une variable d'état entière (`etatCourant`) détermine lequel des panneaux est affiché à chaque itération. Lorsque l'état passe à `ETAT_JOUER`, le thread du menu s'arrête proprement après avoir lancé le thread de jeu.

**Personnaliser le nom de la ville** Le bouton **Modifier le nom** ouvre un panneau secondaire permettant à l'utilisateur de définir le nom de sa ville avant de lancer la simulation, tel qu'illustré en figure 9. Un champ de saisie libre est mis à disposition, pré-rempli avec la valeur par défaut "MA VILLE". Un bouton **Valider et retour au menu** confirme le choix saisi et renvoie l'utilisateur au menu principal. Le nom ainsi défini est affiché en en-tête du tableau de bord latéral tout au long de la simulation.

**Consulter les règles** Le bouton **Règles du jeu** affiche un panneau informatif récapitulant les mécaniques fondamentales de la simulation comme présenté en figure 10. On y retrouve notamment le but du jeu, les seuils critiques des barres de vies (0–25 : maladie physique ou psychologique), les conditions de décès d'un Sim (physique et mental sous 25% pendant trois jours consécutifs), la condition de fin de partie (trois décès), les règles de déplacement manuel, le fonctionnement des relations amicales ainsi que la logique de progression par niveaux. Un bouton **Retour** permet de revenir au menu principal.

**Architecture de navigation** L'ensemble de la navigation entre ces trois panneaux est géré par un `CardLayout` contenu dans un conteneur unique. Ce choix technique garantit que la fenêtre principale

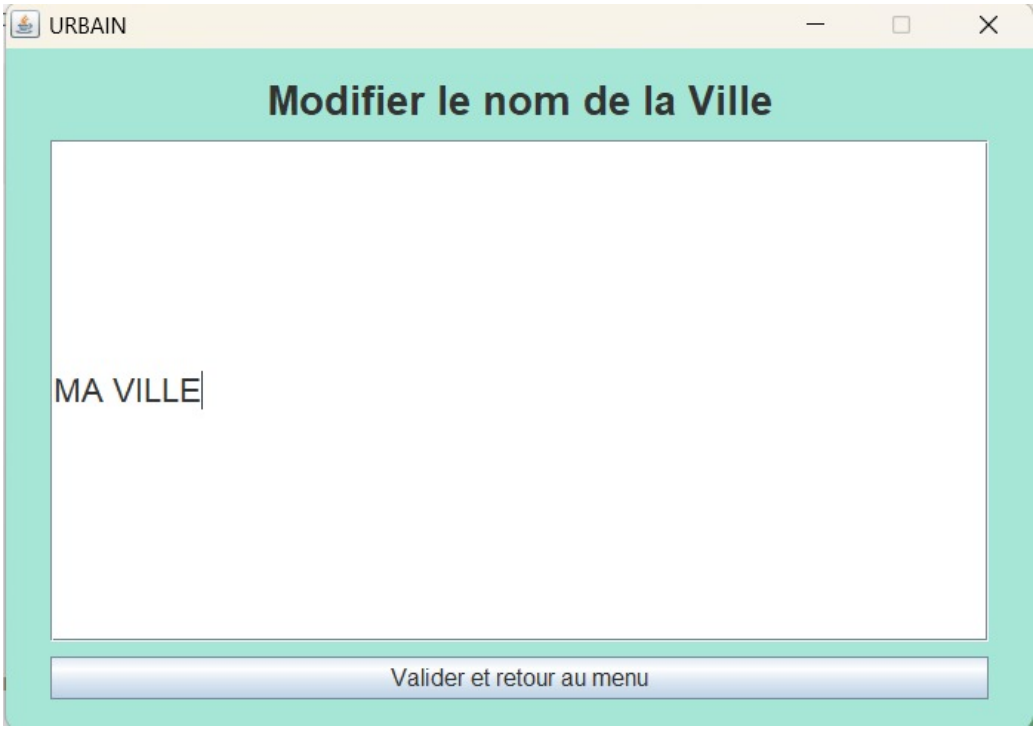



FIGURE 9 – Le panneau de personnalisation du nom de la ville, avec le champ de saisie pré-rempli.

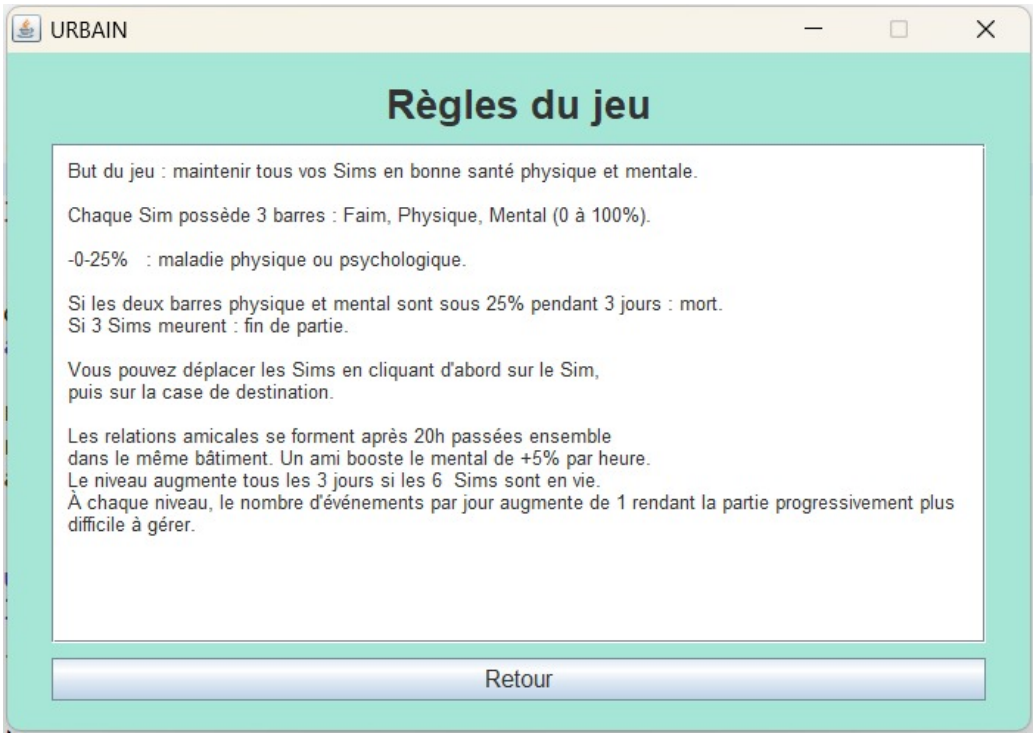



FIGURE 10 – Le panneau des règles du jeu, accessible depuis le menu principal.

na change pas de taille lors des transitions, et que tous les panneaux sont instanciés une seule fois au démarrage, évitant ainsi toute réinstanciation coûteuse lors des allers-retours dans le menu.

## 4.2 Interface de jeu et Dashboard

Une fois la simulation lancée, l'utilisateur accède à la fenêtre principale orchestrée par `MainGUI`. Cette fenêtre repose sur un `BorderLayout` dont le centre est occupé par le rendu graphique de la ville (`GameDisplay`) et la zone de droite par le tableau de bord latéral (`SidePanel`). Ces deux composants sont mis à jour à chaque itération de jeu, cadencé par un `Thread.sleep` correspondant à la vitesse de simulation définie dans `GameConfiguration`. La figure 11 présente une vue d'ensemble de cette fenêtre principale en cours de simulation.




FIGURE 11 – Vue générale de la fenêtre principale au jour 2 : la carte de la ville à gauche avec les Sims visibles, et le tableau de bord latéral à droite affichant les informations de Marie.

**La zone de rendu centrale** La zone rendu est gérée par `GameDisplay`, un `JPanel` dont la méthode `paintComponent` est déclenchée à chaque appel de `repaint()`. Conformément au pattern Strategy mis en peuvre dans les programme du cours, toute la logique de dessin est déléguée à la classe `PaintStrategy`. Cette délégation garantit que `GameDisplay` ne contient aucune logique graphique directe ; il se contente d'appeler successivement les méthodes de dessin de la stratégie. Le rendu s'effectue en plusieurs couches superposées, dans l'ordre suivant :

- La grille de la carte, composée de blocs de taille fixe définie par `GameConfiguration.BLOCK_SIZE`, avec une représentation visuelle des routes, espaces verts et zones bâties sous forme d'images de fond.
- Les bâtiments, représentés par des images distinctives correspondant à leur nature (usine, école, piscine, cinéma, etc.), positionnées selon leurs coordonnées et leurs dimensions (`largeur` et `longueur`) définies dans `Ville`.
- Les Sims, représentés par des sprites distinctifs positionnés sur la case correspondant à leur `positionCourante` et accompagnés de leur nom affiché sous l'icône. Un encadré de surbrillance différencie le Sim actuellement sélectionné par l'utilisateur.

L'intéraction à la souris sur `GameDisplay` est gérée par un `MouseListener`. Un premier clic sur la case occupée par un Sim provoque sa sélection : son nom est transmis au `SidePanel` via la méthode `setSelectedSim()`, déclenchant une mise à jour immédiate du tableau de bord. Un second clic sur une

case vide déplace le Sim sélectionné vers cette destination, court-circuitant temporairement sa routine automatique

**Le tableau de bord latéral** Le `SidePanel` concentre l'ensemble des informations relatives au Sim sélectionné et à l'état général de la simulation. Il se compose de plusieurs zones distinctes, mises à jour à chaque appel de la méthode `nextRound()`. La figure 12 présente le tableau de bord après sélection de Marie, avec la zone des relations amicales renseignée.




FIGURE 12 – Vue de la fenêtre principale avec Marie sélectionnée : le `SidePanel` affiche sa routine, ses relations amicales en cours de formation et le journal des événements.

Les différentes zones du `SidePanel` sont les suivantes :

- **L'en-tête :** le nom de la ville, le numéro du jour courant et le niveau actuel sont affichés en haut du panneau, suivis de l'horloge au format `HH:MM:SS` fournie par le `Chronometer`.
- **L'état global de la ville et du Sim sélectionné :** deux `JProgressBar` représentent respectivement la moyenne des scores globaux de tous les Sims vivants, calculée par `EtatSimManager.updateEtatGen` et le score individuel du Sim sélectionné. Un clic sur la barre du Sim ouvre une fenêtre de détail affichant ses trois jauges individuelles (Faim, Physique, Mental) en temps réel, comme illustré en figure 13.
- **La routine :** le planning horaire du Sim sélectionné est affiché directement dans le `SidePanel`, indiquant pour chaque tranche horaire le bâtiment de destination tel que défini dans sa `Routine` de semaine ou de week-end selon le jour courant.
- **Les relations amicales :** une zone de texte (`JTextArea`) affiche, pour le Sim sélectionné, l'état de chacune de ses relations sociales telles qu'enregistrées dans le `RelationsRepository`. Si aucune relation n'existe, cette zone reste vide. Dans le cas contraire, chaque relation est présentée sous la forme `NomAutre : Nh/20h` tant que le seuil d'amitié n'est pas atteint, puis sous la forme `Amis avec NomAutre depuis Nh` une fois la relation formalisée. La figure 12 illustre ce cas avec les relations de Marie.
- **Le journal des événements et les alertes :** une `JTextArea` défilante affiche les dernières notifications générées par `EvenementManager` et `EtatSimManager`, permettant à l'utilisateur de suivre l'historique des aléas survenus durant la simulation. Un bouton **Alertes Sims en cours** ouvre une fenêtre récapitulant l'ensemble des états actifs de tous les Sims vivants à l'instant courant. La figure 14illustre cette boîte de dialogue au jour 3 : aucun Sim n'étant en état critique, le message « Aucune alerte Sim en cours » est affiché. On peut également observer sur cette capture le `SidePanel` de Marie avec ses relations amicales partiellement visibles (« Amis

avec Julie depuis 5h »), ainsi que le bouton **Recommencer le jeu** en bas du panneau.

— **Le bouton Recommencer le jeu :** situé en bas du `SidePanel`, ce bouton permet à l'utilisateur de réinitialiser entièrement la simulation et de revenir au menu principal, sans avoir à fermer et relancer l'application manuellement.




FIGURE 13 – Fenêtre de détail des barres de vie de Marie : Faim à 98 %, Physique à 94 % et Mental à 86 %.




FIGURE 14 – Fenêtre des alertes Sims en cours au jour 3, niveau 2 : aucune alerte active. On distingue en arrière-plan le `SidePanel` de Marie avec sa routine, ses relations amicales et le bouton **Recommencer le jeu**.

## 4.3 Interaction et gestion des Sims

Cette section détaille les mécanismes mis à la disposition de l'utilisateur pour interagir avec les agents de la simulation.

**Sélection d'un Sim** L'utilisateur peut obtenir des informations précises sur un habitant en effectuant un clic gauche directement sur son icône dans la carte. Cette action déclenche deux mécanismes simultanés :

— **Retour visuel :** La `PaintStrategy` applique un encadré violet autour du Sim sélectionné, le distinguant clairement des autres agents sur le `GameDisplay`. Lorsque l'utilisateur initie une commande de déplacement, l'encadré devient rouge, confirmant que le Sim est prêt à recevoir ses nouvelles coordonnées de destination.
— **Mise à jour du SidePanel :** Le panneau latéral affiche dynamiquement le nom du Sim, l'état de ses trois jauges de besoins (Faim, Physique, Mental) via des `JProgressBar`, ainsi que sa routine actuelle et le détail de ses relations amicales extraites du `RelationsRepository`.

**Déplacement manuel** Bien que le cycle de vie soit automatisé via le `MobileSimManager`, l'utilisateur dispose d'un mode de contrôle manuel pour gérer les situations d'urgence :

1. Sélectionner un Sim par un premier clic sur son icône.
2. Effectuer un second clic sur une case (`Block`) de destination.
3. Le logiciel déplace instantanément l'agent vers les coordonnées cibles, court-circuitant temporairement sa routine automatique.

Ce mécanisme est particulièrement utile lorsqu'un Sim doit être acheminé d'urgence à l'hôpital ou éloigné d'une zone à risque.

**Suivi des événements :** La zone dédiée aux événements dans le `SidePanel` filtre les notifications pour n'afficher que celles relatives au Sim sélectionné. Cela permet à l'utilisateur de comprendre l'origine d'une baisse de moral, notamment liée à un évènement ou à sa routine, et d'adapter sa stratégie en conséquence. Un bouton **Alertes Sims en cours** offre par ailleurs une vue globale de l'état de tous les Sims vivants à l'instant courant.

# 5 Déroulement du projet

Dans cette section, nous décrivons comment le projet a été réalisé en équipe : la répartition des tâches, la synchronisation du travail en membres de l'équipe, etc.

## 5.1 Réalisation du projet par étapes

La réalisation du projet s'est effectuée de manière progressive, en suivant plusieurs points d'avancement définis au fil des semaines. Cette organisation nous a permis de structurer le développement logiciel, de distinguer clairement les différentes couches de l'application (données, traitement et interface graphique), puis d'ajouter progressivement les fonctionnalités prévues dans les spécifications.

**Point d'avancement n°1 (12 février 2026) : Mise en place des bases du projet**
La première étape : a consisté à développer une version simplifiée du jeu afin de mettre en place les fondations techniques du projet. Cette version comprenait :

- Une carte simple avec tout les bâtiments.
- Un personnage (Sim) capable de se déplacer.
- Un déplacement basique permettant au personnage de parcourir automatiquement tout les bâtiments présents sur la carte.

L'objectif principal de cette phase était de valider l'architecture générale du projet ainsi que les échanges entre les différentes couches logicielles.

**Point d'avancement n°2 (16 mars 2026) : Ajout des principales mécaniques**
Lors de la deuxième phase, nous avons Cette version comprenait :

- La réalisation des six Sims.
- La mise en place des trois barres d'état de chaque Sim.
- La diminution progressive de ces besoins au fil du temps.
- Une routine spécifique propre à chaque personnage.
- L'ajout d'un panneau latéral affichant l'heure, le jour, le nom du Sim ainsi que son état global.
- Un système de déplacement manuel à la souris.
- L'évolution des barres d'état selon le temps et les interactions réalisées.
- L'agrandissement de la carte avec de nouvelles dimensions.
- Une première version des différents sprites du jeu a l'aide planner 5D.

Cette étape a permis d'obtenir une simulation plus dynamique et plus proche du résultat attendu.

**Point d'avancement n°3 (12 avril 2026) : Finalisation du contenu du jeu**
La troisième phase correspond à l'intégration des fonctionnalités finales du projet. Le jeu devenait alors presque complet. Nous avons ajouté :

- Le lancement d'une partie, avec la possibilité de modifier le non de la ville et de afficher les règles du jeu ;
- Les relations amicales entre les Sims ;
- L'état général de tout les sims ;
- Des événements aléatoires avec l'affichage dans le sidePannel ;
- L'affichage de la routine en cours ;
- Des améliorations des sprites réalisé dans le dernier point d'avancement ;
- Un système de rassemblement.
- L'affichage de l'état global de la ville.
- Un panneau d'affichage des événements.

Cette phase représentait la version principale du projet avec l'ensemble des fonctionnalités essentielles.

**Rendu final (27 avril 2026) : Tests, corrections et conclusion**
La dernière étape a été consacrée aux tests et aux corrections finales avant le rendu. Plusieurs anomalies détectées durant les essais ont été corrigées.Parmi les ajustements réalisés :

- Ajout de la mort et de la gestion du deuil.

— Mise en place des logs et des tests unitaires afin de détecter et corriger plusieurs problèmes.
— Correction de certaines erreurs identifiées grâce aux tests, ainsi que de certains bâtiments mal configurés.
— Réalisation du rapport de projet.

Cette progression par étapes nous a permis de développer le projet de manière efficace, jusqu'à obtenir une application fonctionnelle et conforme aux objectifs fixés dans le cahier des charges.

**Outils de développement :**
Les outils mobilisés tout au long du développement sont récapitulés dans le liste7. Ils couvrent l'ensemble des dimensions du projet : programmation, conception visuelle, rédaction et collaboration.

1. Langage de programmation : Java
2. IDE : Eclipse
3. Outil de rédaction : Latex
4. Conception des sprites des bâtiments : Planner 5D
5. Gestion et partage du code : Git et GitHub
6. Réalisation des schémas d'architecture logicielle et des diagrammes de classes : LucidChart
7. Création des présentation : Canva

## 5.2 Répartition des tâches, communication et intégration

La répartition du travail s'est faite de manière fonctionnelle : chaque membre était responsable de certaines fonctionnalités, et à chaque point d'avancement, un bilan des tâches à accomplir pour la prochaine étape était établi afin de faire progresser le projet de façon efficace et continue.

Le travail s'est organisé par cycles successifs, proche d'une organisation agile simplifiée, sans méthodologie formelle . Les fonctionnalités étaient développées progressivement, testées, puis améliorées au fil des retours et des besoins du projet. Cette approche a permis une bonne dynamique de production et une adaptation rapide aux problèmes rencontrés.

Le tableau 1 met également en évidence que certaines tache en commun, comme les tests, les corrections ou l'intégration finale, ont nécessité une participation plus large de l'équipe afin d'assurer la cohérence générale du projet. Dans certains cas, lorsque les tâches étaient plus complexes ou volumineuses, un travail en binôme pouvait être mis en place, par exemple pour la réalisation des événements, bien que la majorité du développement ait été réalisée individuellement.

La communication entre les membres de l'équipe s'est faite principalement via WhatsApp, permettant de suivre l'avancement de chacun, de signaler les problèmes et de coordonner les différentes tâches.

Il n'y avait pas de chef de projet désigné. Chaque membre était responsable de ses fonctionnalités, tout en restant attentif à la cohérence globale du projet et au travail des autres membres.

Concernant l'intégration du code, elle a été relativement fluide. Étant donné que les fonctionnalités étaient bien séparées, les conflits lors des fusions Git étaient rares, ce qui a facilité l'intégration progressive du projet.

| **Responsable** | **Tâches principales réalisées** |
|---|---|
| Nur CHOUAREF | Agrandissement de la carte, déplacement manuel des Sims, amélioration des sprites, mise en place des logs, état général des Sims, événements aléatoires, panneau d'affichage des événements. |
| Léanore JEAN | Déplacement automatique des Sims, mise en place des trois barres d'état, panneau latéral, lancement de la partie, relations entre les Sims, affichage de l'état global de la ville. |
| Mohamed LOUKILI | Mise en place des bâtiments, création de la routine des Sims, première version des sprites du jeu, affichage de la routine en cours,ajout de la mort et gestion du deuil. |
| Travail collectif | Evolution des barres d'état selon le temps, correction des bugs, relecture et finalisation du projet. |

Table 1 – Répartition des tâches entre les membres de l'équipe

# 6 Tests du projet

Cette section présente la stratégie de test adoptée pour valider le bon fonctionnement du logiciel Urbain. Les tests sont organisés en quatre catégories : les tests d'utilisation, couvrant les fonctionnalités accessibles à l'utilisateur, les tests de robustesse, vérifiant la stabilité face à des comportements imprévus, les tests de performance et les tests automatisés (unitaires et de performance), destinés à valider le moteur de simulation de manière systématique.

### Tests d'utilisation

Les tests d'utilisation résumé dans le tableau2 vérifient que chaque fonctionnalité de l'interface et de la simulation est accessible et se comporte comme attendu du point de vue de l'utilisateur. Ces tests ont été réalisés manuellement à chaque point d'avancement.

| **Fonctionnalité testée** | **Résultat attendu** | **Résultat observé** |
|---|---|---|
| Affichage du menu | Menu avec boutons Jouer, Modifier le nom et Règles actifs | Menu correct, tous les boutons répondent |
| Déplacement manuel d'un Sim | Le Sim se déplace vers la case cible et quitte sa routine | Déplacement effectué, routine court-circuitée |
| Mise à jour du SidePanel | Le SidePanel affiche les informations du nouveau Sim | Mise à jour instantanée observée |
| Relations amicales dans le SidePanel | Format «Nom : Nh/20h» puis «Amis avec Nom depuis Nh» | Format correct dans les deux états |
| Alertes d'événements | Sims critiques affichés ; «Aucune alerte» sinon | Comportement correct dans les deux cas |
| Changement de routine semaine / week-end | Routine et déplacements correspondent au type de jour | Changement correct au passage du samedi |
| Passage au niveau suivant | Niveau s'incrémente après 3 jours sans décès | Vérifié |
| Fin de partie à 3 décès | Simulation terminée et utilisateur notifié | Vérifié |

Table 2 – Tests d'utilisation

### Tests de robustesse

Au-delà des fonctionnalités nominales, plusieurs scénarios de robustesse ont été identifiés et testés pour vérifier la stabilité du logiciel face à des comportements utilisateur imprévus ou à des situations extrêmes.

| **Scénario testé** | **Résultat attendu** | **Résultat observé** |
|---|---|---|
| Clics rapides répétés sur une case | Pas de doublon dans la file, pas de crash | Vérifié |
| Déplacement hors des limites de la carte | Déplacement ignoré ou bloqué aux bords | Vérifié |
| Jeu sans interaction sur la durée | Simulation stable, pas de fuite mémoire | Verifié |

Table 3 – Tests de robustesse

**Tests unitaires** Pour couvrir systématiquement l'ensemble des composants du moteur de simulation, dix classes de tests unitaires ont été implémentées et regroupées dans une `TestSuite` commune, couvrant les managers principaux (états, événements, niveau, relations), les entités métier (Evenement, RelationAmicale, Batiment, BoundedCounter) ainsi que la couche de persistance (RelationsRepository).

**Tests de performance** Les tests de performance visent à évaluer la réactivité et la stabilité du moteur du jeu sur le long terme. Ils répondent à trois questions clés : est-ce que la machine ralentit au lancement du jeu ? Combien de temps prend `updateBesoins()` pour les 6 Sims ? Le jeu tient-il sur une semaine complète ? Ces tests sont assurés par la classe `TestPerformance` et ont été exécutés sur machine. La simulation consomme environ 250 Mo de mémoire, e 7% du processeur disponible, ce qui est responsable pour un tel jeu.

| **Test** | **Résultat mesuré** |
|---|---|
| 1000 appels à `updateBesoins()` | 6 ms |
| 1000 appels à `updateEtats()` | 0 ms |
| 100 heures de relations simulées | 10 ms |
| Simulation d'une journée complète (24h) | 2 ms |
| Simulation d'une semaine complète (7j × 24h) | 7 ms |
| 10 000 appels à `updateEtatGeneral()` | 3 ms |
| Consommation mémoire globale | 200Mo (7%) |

TABLE 4 – Tests de performance

# 7 Conclusion et perspectives

## 7.1 Résumé du travail réalisé

Le projet *Urbain* a permis de concevoir et de développer une simulation de vie urbaine fonctionnelle en Java, en mobilisant des concepts avancés de programmation orientée objet et de génie logiciel. Au cours de ce semestre, nous avons mis en place les éléments suivants :

- **Un moteur de simulation robuste :** grâce aux différents managers (`MobileSimManager`, `EtatSimManager`, `EvenementManager`, `NiveauManager`), la simulation gère de manière autonome les routines des Sims, leurs besoins vitaux et leurs interactions sociales. Le mécanisme de sursis à trois jours assure une gestion équitable du cycle de vie.
- **Une architecture découplée :** l'application suit une structure proche du modèle MVC, séparant clairement la logique métier (Repositories et Managers) de l'interface graphique (`MainGUI`, `SidePanel`). L'usage du patron *Singleton* pour les repositories garantit la cohérence des données partagées entre les managers.
- **Une gestion dynamique des aléas :** le système d'événements aléatoires offre une expérience de jeu non linéaire, où l'utilisateur doit constamment réagir pour assurer la survie et le bien-être de la population.
- **Une interface utilisateur soignée :** le dashboard graphique avec ses sprites de bâtiments, ses barres de vie colorées et son journal d'événements rend la simulation accessible et lisible, même lors de situations complexes.

Ce travail nous a également confrontés aux réalités du développement en équipe : gestion rigoureuse des versions avec Git, communication continue, et intégration progressive des modules IHM et moteur sans conflits majeurs.

## 7.2 Améliorations possibles du projet

Bien que le prototype actuel soit fonctionnel et stable, plusieurs axes d'évolution permettraient de transformer cette simulation en un logiciel encore plus complet :

1. **Déplacement fluide :** actuellement, le `MobileSimManager` gère les déplacements de manière discrète, ce qui s'apparente à une téléportation entre deux points de routine. Une amélioration majeure consisterait à implémenter un déplacement fluide case par case, en développant un algorithme de recherche de chemin capable de calculer une trajectoire en temps réel sur la `Map` tout en gérant les collisions avec les bâtiments.
2. **Dynamisme démographique :** pour rendre la simulation plus vivante, un système d'immigration géré par le `NiveauManager` pourrait être introduit. Une fois qu'on aurait atteint un certain niveau, de nouveaux Sims emménageraient dynamiquement, avec leurs propres routines générées à la volée.
3. **Sauvegarde de la partie :** une fonctionnalité essentielle pour un logiciel de simulation est la capacité de sauvegarder et de charger une partie. L'utilisation de la sérialisation Java (`Serializable`) permettrait d'exporter l'état complet des données, relations amicales, statistiques des Sims, jour et heure du `Chronometer`, offrant ainsi une continuité d'expérience à l'utilisateur.
4. **Contrôle de l'échelle temporelle :** l'ajout d'une fonctionnalité permettant d'accélérer la simulation améliorerait significativement l'expérience utilisateur. Cela permettrait d'éviter aux joueurs de rapidement s'ennuyer.
5. **Refactorisation vers un `SimsRepository` :** une amélioration structurelle consisterait à transformer la classe `Sims` en un véritable repository utilisant le patron *Singleton*. Cela permettrait d'uniformiser l'accès aux personnages pour tous les Managers sans faire circuler l'instance dans les constructeurs, simplifiant ainsi l'architecture globale. Cette modification permettrait également d'ajouter une fonctionnalité permettant de changer les noms des personnages.

En conclusion, ce projet a constitué une étape clé dans notre apprentissage du Génie Logiciel. Il nous a permis de transformer des concepts théoriques de conception de systèmes en une application concrète, modulaire et évolutive, tout en nous préparant aux exigences du travail collaboratif en milieu professionnel.